

Employee performance mapping Queries

1. Create a database named employee, then import data_science_team.csv, proj_table.csv and emp_record_table.csv into the employee database from the given resources

Query:

Employee record table

```
create database employee;

use employee;

create table emp_record_table(

EMP_ID varchar(100),

FIRST_NAME varchar(100),

LAST_NAME varchar(100),

GENDER varchar(100),

ROLE varchar(100),

DEPT varchar(100),

EXP int,

COUNTRY varchar(100),

CONTINENT varchar(100),

SALARY int,

EMP_RATING int,

MANAGER_ID varchar(100),

PROJ_ID varchar(100)

);
```

Project table

```
create table Proj_table(

PROJECT_ID varchar(100),

PROJ_Name varchar(100),

DOMAIN varchar(100),

START_DATE date,
```

Employee performance mapping Queries

```
CLOSURE_DATE date,  
DEV_QTR varchar(100),  
STATUS varchar(100)  
);
```

Data science team table

```
create table data_science_team(  
EMP_ID varchar(100),  
FIRST_NAME varchar(100),  
LAST_NAME varchar(100),  
GENDER varchar(100),  
ROLE varchar(100),  
DEPT varchar(100),  
EXP int,  
COUNTRY varchar(100),  
CONTINENT varchar(100)  
);
```

2. Create an ER diagram for the given employee database.

Use GUI for creating ER diagram

3. Write a query to fetch EMP_ID, FIRST_NAME, LAST_NAME, GENDER, and DEPARTMENT from the employee record table, and make a list of employees and details of their department.

Employee performance mapping Queries

Query

```
select EMP_ID, FIRST_NAME, LAST_NAME, GENDER, DEPT  
from emp_record_table;
```

```
SELECT EMP_ID, FIRST_NAME, LAST_NAME , DEPT  
FROM emp_record_table;
```

4. Write a query to fetch EMP_ID, FIRST_NAME, LAST_NAME, GENDER, DEPARTMENT, and EMP_RATING if the EMP_RATING is: ● less than two ● greater than four ● between two and four.

Query

```
SELECT  
    EMP_ID,  
    FIRST_NAME,  
    LAST_NAME,  
    GENDER,  
    DEPT,  
    EMP_RATING  
FROM emp_record_table  
WHERE EMP_RATING < 2  
    OR EMP_RATING > 4  
    OR EMP_RATING BETWEEN 2 AND 4;
```

5. Write a query to concatenate the FIRST_NAME and the LAST_NAME of employees in the Finance department from the employee table and then give the resultant column alias as NAME.

Query

```
SELECT  
    CONCAT(FIRST_NAME, ' ', LAST_NAME) AS NAME  
FROM emp_record_table  
WHERE DEPT = 'Finance';
```

6. Write a query to list only those employees who have someone reporting to them. Also, show the number of reporters (including the President).

Query

Employee performance mapping Queries

```
SELECT
    M.EMP_ID,
    M.FIRST_NAME,
    M.LAST_NAME,
    COUNT(E.EMP_ID) AS NUMBER_OF_REPORTERS
FROM emp_record_table M
JOIN emp_record_table E
    ON M.EMP_ID = E.MANAGER_ID
GROUP BY
    M.EMP_ID, M.FIRST_NAME, M.LAST_NAME;
```

7. Write a query to list down all the employees from the healthcare and finance departments using union. Take data from the employee record table.

Query

```
SELECT EMP_ID, FIRST_NAME, LAST_NAME, DEPT
FROM emp_record_table
WHERE DEPT = 'Healthcare'
```

UNION

```
SELECT EMP_ID, FIRST_NAME, LAST_NAME, DEPT
FROM emp_record_table
WHERE DEPT = 'Finance';
```

8. Write a query to list down employee details such as EMP_ID, FIRST_NAME, LAST_NAME, ROLE, DEPARTMENT, and EMP_RATING grouped by dept. Also include the respective employee rating along with the max emp rating for the department.

Query

```
SELECT
    EMP_ID,
    FIRST_NAME,
    LAST_NAME,
    ROLE,
    DEPT,
    EMP_RATING,
    MAX(EMP_RATING) OVER (PARTITION BY DEPT) AS MAX_DEPT_RATING
FROM emp_record_table;
```

9. Write a query to calculate the minimum and the maximum salary of the employees in each role. Take data from the employee record table.

Employee performance mapping Queries

```
Quary
SELECT
    ROLE,
    MIN(SALARY) AS MIN_SALARY,
    MAX(SALARY) AS MAX_SALARY
FROM emp_record_table
GROUP BY ROLE;
```

10. Write a query to assign ranks to each employee based on their experience. Take data from the employee record table.

```
Quary
SELECT
    EMP_ID,
    FIRST_NAME,
    LAST_NAME,
    EXP,
    RANK() OVER (ORDER BY EXP DESC) AS EMP_RANK
FROM emp_record_table;
```

11. Write a query to create a view that displays employees in various countries whose salary is more than six thousand. Take data from the employee record table.

```
Quary
CREATE VIEW high_salary_employees AS
SELECT
    EMP_ID,
    FIRST_NAME,
    LAST_NAME,
    COUNTRY,
    SALARY
FROM emp_record_table
WHERE SALARY > 6000;
```

12. Write a nested query to find employees with experience of more than ten years. Take data from the employee record table.

```
Quary
SELECT
    EMP_ID,
    FIRST_NAME,
    LAST_NAME,
    EXP
FROM emp_record_table
WHERE EMP_ID IN (
```

Employee performance mapping Queries

```
SELECT EMP_ID
FROM emp_record_table
WHERE EXP > 10
);
```

13. Write a query to create a stored procedure to retrieve the details of the employees whose experience is more than three years. Take data from the employee record table.

Quary

```
DELIMITER //
```

```
CREATE PROCEDURE GetExperiencedEmployees()
BEGIN
    SELECT
        EMP_ID,
        FIRST_NAME,
        LAST_NAME,
        EXP
    FROM emp_record_table
    WHERE EXP > 3;
END //
```

```
DELIMITER ;
```

```
SELECT * FROM emp_record_table WHERE EXP > 3;
```

14. Write a query using stored functions in the project table to check whether the job profile assigned to each employee in the data science team matches the organization's set standard.

Quary

```
DELIMITER //
```

```
CREATE FUNCTION CheckJobProfile(job_profile VARCHAR(50))
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    IF job_profile = 'Data Scientist' THEN
        RETURN 'Match';
    ELSE
```

Employee performance mapping Queries

```
        RETURN 'Not Match';
    END IF;
END //
```



```
DELIMITER ;
```



```
SELECT
    EMP_ID,
    PROJECT_NAME,
    TEAM,
    JOB_PROFILE,
    CheckJobProfile(JOB_PROFILE) AS STATUS
FROM proj_table
WHERE TEAM = 'Data Science';
```

15. Create an index to improve the cost and performance of the query to find the employee whose FIRST_NAME is 'Eric' in the employee table after checking the execution plan.

Query

```
EXPLAIN SELECT *
FROM emp_record_table
WHERE FIRST_NAME = 'Eric';
```

```
CREATE INDEX employee_first_name
ON emp_record_table(FIRST_NAME);
```

```
EXPLAIN SELECT *
FROM emp_record_table
WHERE FIRST_NAME = 'Eric';
```

```
CREATE INDEX employee_fname_dept
ON emp_record_table(FIRST_NAME, DEPT);
```

16. Write a query to calculate the bonus for all the employees, based on their ratings and salaries (Use the formula: 5% of salary * employee rating).

Query

```
SELECT
    emp_id,
    first_name,
```

Employee performance mapping Queries

```
    salary,  
    emp_rating,  
    (0.05 * salary * emp_rating) AS bonus  
FROM emp_record_table;
```

17. Write a query to calculate the average salary distribution based on the continent and country. Take data from the employee record table.

Query

```
SELECT  
    continent,  
    country,  
    AVG(salary) AS avg_salary  
FROM emp_record_table  
GROUP BY continent, country  
ORDER BY continent, country;
```